

A Game-Development Paradigm for Building Programming Intuition

William J. Joel

Computer Science Department, Western Connecticut State University, Danbury, CT USA

joelw@wcsu.edu

ABSTRACT

A fundamental aspect of learning computer programming is the development of an intuition that can provide one with what needs to be done to solve a problem sans specific implementation detail. However, intuition cannot be taught using just lectures and/or readings. Students must be immersed in the process of programming, so relationships among the elements they learn become apparent. Often, short, discrete demonstration programs are used to introduce various programming schema. Given the disconnectedness of these short examples, they fail to help students gain a programming intuition. Using game development as the basis for an introductory programming class taps into students existing understanding of and intuition relating to image, sound, and games. It also demonstrates how various programming approaches can be used to the benefit of solving a large, meaningful problem, which, appropriately designed, provides ample space for the development of a student's programming intuition.

CCS CONCEPTS

• **Applied computing** → Education; Collaborative learning; Computers in other domains; Personal computers and PC applications; Computer games.

KEYWORDS

Introduction to computing, Applied computing, Education, Collaborative learning

ACM Reference Format:

William J. Joel. 2022. A Game-Development Paradigm for Building Programming Intuition. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Educator's Forum (SIGGRAPH '22 Educator's Forum)*, August 07–11, 2022, Vancouver, BC, Canada. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3532724.3535601>

1 OVERVIEW

One of the issues with teaching computer programming to students is the rate at which they develop a programming intuition. At some point during their evolution as programmers, it becomes essential to develop an intuitive sense of what to do at a given point in software development, and how to efficiently and effectively locate and correct programming errors. [Bonar and Soloway, 1983] However, the types of practice exercises students are most often

presented with are often discrete in nature. Abstract, intangible exercises force students to visualize relationships amount various data without the benefit of tapping into real-world experiences. In addition, these exercises occasionally ask students to not only learn new programming concepts, but to become acquainted with new, unfamiliar algorithms. Van Loan [Van Loan, 2021] states that “intuition is a sense of direction, essentially no different from the sense of direction that enables you to find your way around an old childhood neighborhood without a map. The key is that you’ve been there before. “If intuition is a sense of direction, then computational intuition is a sense of computational direction.” Over the past several years, to foster a sense of “computational direction,” I have retooled my approach to teaching our department’s introductory programming class. Disjoint practice exercises were replaced with a single, semester-long exercise that focuses on the manipulation of image and sound, and game development.

Rather than presenting students with discrete programming exercises, my course is organized about the development of a single computer game. During the semester, the students work together to create a simple, 2D, arcade-style game. New game elements are added after students are presented with related, requisite programming strategies and concepts. “Games can be a valuable tool for enriching computer science education, since they can facilitate a number of conditions that promote learning: student motivation, active learning, adaptivity, collaboration, and simulation.” [Johnson et al., 2016] Soares [Soares, 2011] presents a problem-based learning (PBL) course design where the final project is a computer game using Java. During Soares’ course, students were introduced to Object-Oriented programming (OOP) and Graphical User Interface programming (GUI). However, unlike Soares, who included game development as a final project, my course design fully embraces game development as the backbone for learning. Goulding and DiTrollo [Goulding and DiTrollo, 2007] discuss an introductory course that uses game development as a major element. However, this course uses “. . . classroom methods which favor independent study, cooperative learning and teamwork over lectures and individual achievement.” As such, their course has more the “look & feel” of independent studies rather than a standard lecture course. Holliday [Holliday, 1995] presents a course structure similar to mine. “[T]he enthusiasm students have for playing computer games can be used in an introductory programming course to increase the enthusiasm and attention that students have for developing problem solving and programming skills.” However, given the early date of this paper, it’s odd that more have not used this approach.

In my course design, students see the use of all programming elements typically presented in an introductory course, though the order they are presented in may vary from semester to semester. Students are shown the importance of each concept they are asked

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH '22 Educator's Forum, August 07–11, 2022, Vancouver, BC, Canada

© 2022 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9366-9/22/08.

<https://doi.org/10.1145/3532724.3535601>

to explore, given a clearly defined rationale for the use of each programming technique based on game requirements.

2 COURSE ORGANIZATION

2.1 Game Design

After a few, introductory sessions, students are told that all programming examples during the term will represent steps in the development of a 2D, arcade-style computer game. Initially, students are assigned to small (4-5 students) groups to discuss game design. They are also presented with a list of Do's and Don'ts.

- Game is strictly 2D
- All game elements (hero, villain, props, etc.) are displayed using simple bitmaps (sprites)
- Bitmaps should be acquired rather than created
- Different bitmaps used to represent a given game element
- No animation of game elements is allowed
- Sounds must be simple, if used at all
- Game rules must be simple
- Only one game level
- Player attains a "win" with no more than moderate effort
- Graphics and sound libraries may be used
- No game engine may be employed

Each team's game design is shared with the entire class. The entire class votes on which game design to adopt. However, I have the right to modify the design to ensure that the game project can be completed during the semester.

2.2 Topic Sequencing

The order in which various programming concepts are introduced to students is strictly governed by the chosen game design as shown in Table 1. After selecting a game design, I review the design to determine the order with which students are presented with new programming concepts. For example, games often represent infinite loops. Programs developed with programming languages such as Processing are, by their nature, infinite loops. For other languages, such as Python, a brief introduction, to while loops, becomes necessary. Later in the semester, a more complete discussion of loops can be presented as required. Decision statements (If-Then-Else) are control structures necessary to implement keyboard controls, and collision detection and response. This use of decision statements allows students to implement a rudimentary game engine, one that is much simpler than engines commonly used in game development. An exploration of decision statements in games also lends itself to a discussion of flowcharts.

3 DISCUSSION

PBL is a paradigm that presents a problem to students and then instructs them in concepts required to develop a solution. However, after a cursory review of relevant literature, the use of game development as the major pedagogical focus for an introductory programming class appears to be relatively unique, with few other examples of this specific approach. By first presenting a new problem, students must analyze the problem to determine what they already know and where their respective skill sets are lacking. The use of PBL helps to avoid questions such as, "Why do I need to learn

this?" By presenting a problem first, students can clearly see why they need to learn what will subsequently be presented in class. By asking students to develop a program that works with images and sounds, they can tap into pre-existing intuitions. To be successful, students need to think like computer programmers, to adopt the mental schemas utilized by programmers. Thus, any strategy that facilitates intuition's acquisition is well worth the effort.

Table 1: Game Components & Programming Concepts

Game Components	Programming Concepts
Image display	Graphics API
Element attributes	Variables
	Assignment
Element movement	While loops
Collision detection & response	Decision statements
	Arrays
	For loops
Multiple copies	Arrays
	For loops
Saved game statistics	Text file input/output
Element templates	Classes & objects

4 CONCLUSIONS

As the various computer-based fields continue to expand, it becomes critical to find new and better ways to not only teach the fundamentals of computer programming, but also foster the development of schema for program design intuition. By using existing visual and auditory intuition, an introductory programming course has been developed that uses game development to facilitate the acquisition of a programming intuition.

REFERENCES

- Jeffrey Bonar and Elliot Soloway. 1983. Uncovering principles of novice programming. *POPL '83: Proceedings of the 10th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, January 1983, pp 10–13.
- Charles Van Loan. 2021. Building Freshman Intuition for Computational Science and Mathematics. SIAM News. Online: <https://cpb-us-e1.wpmucdn.com/blogs.cornell.edu/dist/c/9924/files/2021/11/FreshmanIntuition.pdf>
- Chris Johnson, Monica McGill, Durell Buchard, Michael K. Bradshaw, Victor A. Bucheli, Laurence D. Merkle, Michael James Scott, Z. Sweedyk, and J. Ángel. 2016. Game Development for Computer Science Education. *ITiCSE '16: Proceedings of the 2016 ITiCSE Working Group Reports*, July 2016, pp 23–44.
- Andrey Soares. 2011. Problem based learning in introduction to programming courses. *Journal of Computing Sciences in Colleges*, 27(1), October 2011, pp. 36.
- Tom Goulding and Rita DiTrollo. 2007. Complex game development by freshman computer science majors. *ACM SIGCSE Bulletin*, 39(4), December 2007, pp 92–99.
- Mark A. Holliday. 1995. Incremental game development in an introductory programming course. *Proceedings of the 33th Annual Southeast Regional Conference*, March, 1995, pp 170–175.